INT1339

Ngôn ngữ Lập trình C++ Cơ Sở

Giảng viên: Lê Hà Thanh

lehathanh@ptithcm.edu.vn

Nội dung

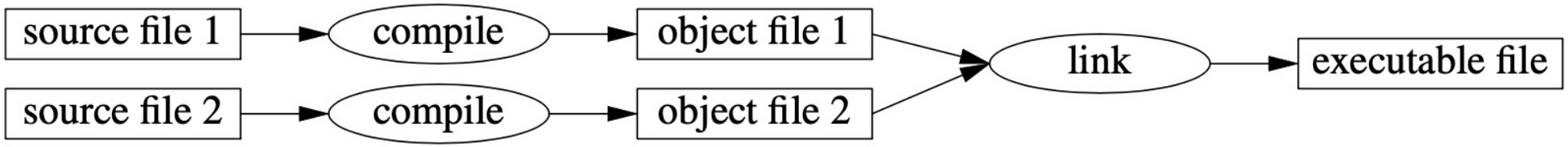
1. Giới thiệu
2. C++ thiết lập môi trường
3. C++ cú pháp cơ sở
4. C++ các kiểu dữ liệu
5. Phạm vi truy cập biến trong C++
6. C++ hằng số
7. Các toán tử trong C++
8. C++ loops - Vòng lặp
9. C++ functions - hàm
10. C++ arrays - mảng
11. C++ strings - chuỗi
12. C++ pointers - con trỏ
13. C++ references - tham chiếu
14. C++ các phép xuất nhập cơ sở
15. C++ structure – Kiểu Cấu trúc
16. C++ Class – Kiểu lớp
17. Xử lý lỗi trong C++

Giới thiệu

- C++ là ngôn ngữ lập trình trên nhiều nền tảng phổ biến nhất
- Được dùng để tạo các ứng dụng và phần mềm hiệu suất cao như HĐH, Trò chơi, phần mềm thương mại điện tử, v.v.
- Được phát triển bởi Bjarne Stroustrup, như một phần mở rộng của ngôn ngữ C.
- C++ kiểm soát rất tốt tài nguyên hệ thống và bộ nhớ.
- *C, C++, Java, C# có cấu trúc cú pháp (syntax) tương tự*

Giới thiệu

- C++ là ngôn ngữ biên dịch (compile)
- Để một chương trình có thể chạy, văn bản nguồn của nó phải được xử lý bằng trình biên dịch, tạo ra các tệp đối tượng, được kết hợp bởi một trình liên kết để tạo ra một chương trình thực thi được.
- Một chương trình C++ bao gồm nhiều tệp mã nguồn (thường được gọi đơn giản là tệp nguồn).



Thiết lập Môi trường

- Một chương trình thực thi được tạo cho sự kết hợp phần cứng/hệ thống cụ thể;
  - chẳng hạn, chương trình không thể chạy được từ máy Mac sang PC Windows.
- Mã nguồn chương trình C++ có thể được biên dịch thành công và chạy trên nhiều hệ thống khác nhau.

Thiết lập Môi trường

- Tiêu chuẩn ISO C++ quy định hai thành phần:
  - Phần ngôn ngữ cốt lõi, chẳng hạn như các kiểu dựng sẵn (ví dụ: *char* và *int*) và các vòng lặp (ví dụ: câu lệnh *for* và câu lệnh *while*)
  - Các thành phần thư viện chuẩn, chẳng hạn như các *container* (ví dụ: *vector* và *map*) và các thao tác I/O (ví dụ: << và getline())
- C++ là một ngôn ngữ tĩnh.
  - Loại (type) của mọi thực thể (ví dụ: đối tượng, giá trị, tên và biểu thức) phải được trình biên dịch biết tại thời điểm sử dụng.
  - Kiểu của một đối tượng xác định tập hợp các thao tác áp dụng cho nó.

C++ Cú pháp Cơ sở

- **int main() { }** // the minimal C++ program

- Hello, World!

```
1.  #include <iostream>
2.  int main() {
3.      std::cout << "Hello, World!\n";
4.  }
```

- Trình biên dịch dùng cơ chế luồng xuất nhập (I/O) chuẩn trong thư viện **iostream**
- **std::cout** : luồng xuất chuẩn
- **<<** : gửi chuỗi (hằng) ra

C++ Cú pháp Cơ sở

```
1.  #include <iostream>
2.  using namespace std; // make names from std visible without std::
3.  double square(double x) // square a double precision floating-point number
4.  {
5.      return x*x;
6.  }
7.  void print_square(double x)
8.  {
9.      cout << "the square of " << x << " is " << square(x) << "\n";
10. }
11. int main()
12. {
13.     print_square(1.234); // print: the square of 1.234 is 1.52276
14. }
```

Kiểu trả về **void** cho biết hàm không trả về giá trị nào cả.

C++ Các kiểu dữ liệu

- Khai báo tường minh một biến trong chương trình
  <type> <variable>;

int inch;

- Type: Kiểu dữ liệu, định nghĩa tập các giá trị và tập các phép toán (operations) trong kiểu.
- Object: Đối tượng, phần nhớ chứa một giá trị thuộc một kiểu dữ liệu.
- Value: Giá trị, tập hợp các bits được dịch thành giá trị tương ứng với kiểu.
- Variable: Biến, tên được đặt cho đối tượng

C++ Các kiểu dữ liệu

- Có nhiều kiểu dữ liệu cơ bản

1. **bool //** Boolean, possible values are true and false
2. **char //** character, for example, 'a', ' z', and '9'
3. **int //** integer, for example, 1, 42, and 1066
4. **double //** double-precision floating-point number, for example, 3.14 and 299793.0

...

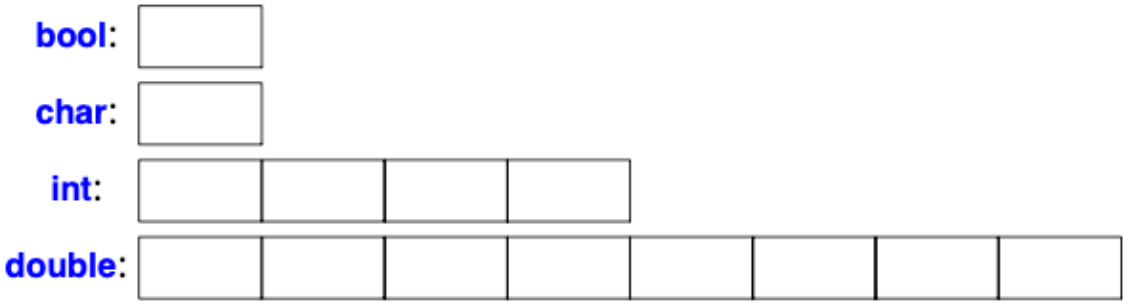
- Kiểu dữ liệu cơ bản phụ thuộc phần cứng, có kích thước cố định xác định phạm vi giá trị có thể được lưu trữ trong đó

C++ Các kiểu dữ liệu – Độ lớn

- Biến **char** giữ giá trị là một ký tự (character), độ lớn 8-bit, hay 1 byte
- Độ lớn các kiểu khác là bội số của một **char**
- Độ lớn của kiểu được khai báo khi cài đặt thực tế (có thể khác nhau tùy loại máy tính)
- Toán tử xác định kích thước kiểu: **sizeof** ,
  ví dụ :

  sizeof(char) cho kết quả =1

  sizeof(int) thường cho kết quả =4



C++ Các Toán tử Số học

1. **x+y** //plus
2. **+x** // unary plus
3. **x–y** //minus
4. **–x** // unary minus
5. **x∗y** // multiply
6. **x/y** //divide
7. **x%y** // remainder (modulus) for integers

C++ Các Toán tử So sánh

1. **x==y //equal**
2. **x!=y // not equal**
3. **x<y // less than**
4. **x>y // greater than**
5. **x<=y // less than or equal**
6. **x>=y // greater than or equal**

C++ Các kiểu dữ liệu – Chuyển đổi Kiểu

- C++ tự động thực hiện *chuyển đổi kiểu* giữa các kiểu cơ bản khi cần thiết

```
1. void some_function()      // function that doesn't return a value
2. {
3.     double d = 2.2;     // initialize floating-point number
4.     int i = 7;          // initialize integer
5.     d = d+i;            // assign sum to d
6.     i = d*i;            // assign product to i (truncating the double d*i to an int)
7. }
```

- Toán tử =: toán tử gán
- Toán tử ==: toán tử so sánh, kiểm tra bằng nhau

C++ Các kiểu Dữ liệu – Khởi tạo Giá trị

```
1. double d1 = 2.3;
2. double d2 {2.3};

3. complex<double> z = 1; // a complex number with double-precision floating-point scalars
4. complex<double> z2 {d1,d2}; // initialize with constructor
5. complex<double> z3 = {1,2}; // the = is optional with { … }

6. vector<int> v {1,2,3,4,5,6}; // a vector of ints
```

C++ Các kiểu Dữ liệu – Gán Kiểu tự động với auto

1. **auto b = true;** // a bool
2. **auto ch = 'x';** // a char
3. **auto i = 123;** // an int
4. **auto d = 1.2;** // a double
5. **auto z = sqrt(y);** // z has the type of whatever sqrt(y) returns

- Dùng auto để
  - Tránh phải ghi tên kiểu tường minh khi lập trình viên khó nhận biết kiểu dữ liệu chính xác của một đối tượng
  - Tên kiểu dài
- Không dùng auto nếu:
  - Khai báo tường minh kiểu dữ liệu cho người đọc code
  - Phân biệt rõ miền giá trị và độ chính xác của biến (ví dụ: phân biệt double với float)

C++ Các toán tử khác

1. **x+=y** //x=x+y
2. **++x** // increment: x = x+1
3. **x−=y** //x=x-y
4. **−−x** // decrement: x = x-1
5. **x∗=y** // scaling: x = x*y
6. **x/=y** // scaling: x = x/y
7. **x%=y** //x=x%y

- Các toán tử này hoạt động chính xác, thuận tiện, rất hay được sử dụng

C++ Hằng số

- **const**: "hứa không thay đổi giá trị này". Để chỉ định giao diện sao cho dữ liệu được chuyển đến các hàm mà không sợ bị sửa đổi.
- **constexpr**: khai báo hằng "được đánh giá tại thời điểm biên dịch". Để chỉ định hằng số, cho phép đặt dữ liệu vào bộ nhớ khó bị hỏng và để đảm bảo hiệu suất.

```
const int dmv = 17;                          // dmv is a named constant
int var = 17;                                // var is not a constant
constexpr double max1 = 1.4*square(dmv);     // OK if square(17) is a constant expression
constexpr double max2 = 1.4*square(var);     // error: var is not a constant expression
const double max3 = 1.4*square(var);         // OK, may be evaluated at run time

double sum(const vector<double>&);           // sum will not modify its argument
vector<double> v {1.2, 3.4, 4.5};            // v is not a constant
const double s1 = sum(v);                    // OK: evaluated at run time
constexpr double s2 = sum(v);                // error: sum(v) not constant expression
```

C++ loops – Kiểm tra bằng Vòng lặp

```
bool accept3()
{
        int tries = 1;
        while (tries<4) {
                cout << "Do you want to proceed (y or n)?\n";        // write question
                char answer = 0;
                cin >> answer;                                       // read answer

                switch (answer) {
                case 'y':
                        return true;
                case 'n':
                        return false;
                default:
                        cout << "Sorry, I don't understand that.\n";
                        ++tries;    // increment
                }
        }
        cout << "I'll take that for a no.\n";
        return false;
}
```

C++ Arrays, Pointers - Mảng, Con trỏ

- [ ]: mảng của các giá trị thuộc <kiểu>

  char v[6]; *// array of 6 characters*

- * : con trỏ tới giá trị thuộc <kiểu>

  char* p; *// pointer to character*

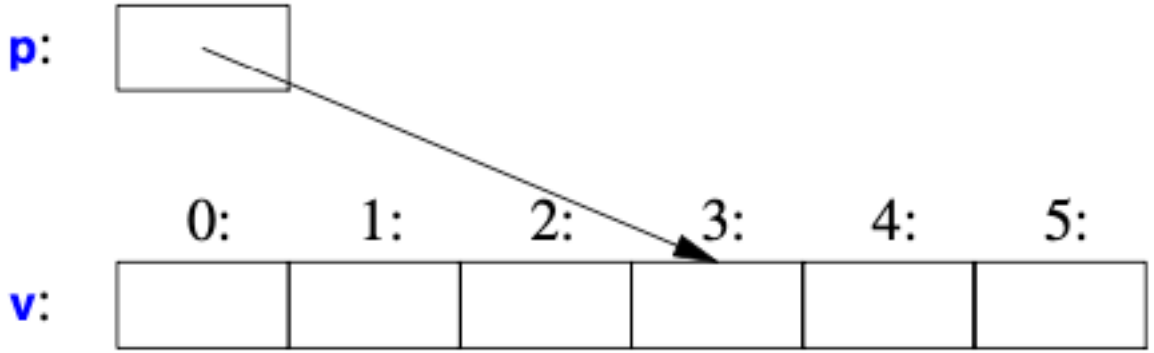
- Phần tử đầu tiên của mảng ở vị trí 0
  - Mảng v có 6 phần tử v[0], v[1], …, v[5]
- Độ lớn của mảng phải là một biểu thức không đổi

C++ Arrays, Pointers - Mảng, Con trỏ

- Biến con trỏ giữ địa chỉ của đối tượng thuộc về kiểu đã khai báo

```
char* p = &v[3]; // p points to v's fourth element
char x = *p;     // *p is the object that p points to
```

- Tiền tố * mang nghĩa "nội dung của"
- Tiền tố & mang nghĩa "địa chỉ của"



C++ Arrays, Pointers - Mảng, Con trỏ và Vòng lặp

- Chép các phần tử mảng sang mảng khác

```
void copy_fct()
{
        int v1[10] = {0,1,2,3,4,5,6,7,8,9};
        int v2[10];                    // to become a copy of v1

        for (auto i=0; i!=10; ++i)   // copy elements
                v2[i]=v1[i];
        // ...
}
```

C++ Arrays, Pointers - Mảng, Con trỏ và Vòng lặp

- Duyệt phần tử mảng dùng khai báo auto
- "Với từng phần tử của v, tính từ đầu đến cuối mảng, chép giá trị của phần tử đó vào x rồi in x ra"
- for tự động duyệt toàn bộ các phần tử mảng

```
void print()
{
        int v[] = {0,1,2,3,4,5,6,7,8,9};

        for (auto x : v)                // for each x in v
                cout << x << '\n';

        for (auto x : {10,21,32,43,54,65})
                cout << x << '\n';
        // ...
}
```

C++ References – Tham chiếu

- Dùng biến tham chiếu đến phần tử mảng
- Tiền tố & có nghĩa “tham chiếu đến”

```
void increment()
{
        int v[] = {0,1,2,3,4,5,6,7,8,9};

        for (auto& x : v)
                ++x;
        // ...
}
```

C++ Mảng, Con trỏ, Tham chiếu

- Các toán tử & , *, [ ] gọi là toán tử khai báo

```
T a[n];     // T[n]: array of n Ts
T* p;       // T*: pointer to T
T& r;       // T&: reference to T
T f(A);     // T(A): function taking an argument of type A returning a result of type T
```

- Đảm bảo con trỏ **luôn** trỏ đến một đối tượng để phép tham chiếu ngược có ý nghĩa.
- Nếu không có đối tượng cụ thể, trỏ đến giá trị **nullptr** ("con trỏ rỗng" – the null pointer)

```
double* pd = nullptr;
Link<Record>* lst = nullptr;    // pointer to a Link to a Record
int x = nullptr;                // error: nullptr is a pointer not an integer
```

C++ Mảng, Con trỏ, Tham chiếu

- Nguyên tắc thực hành: Luôn kiểm tra tính hợp lệ của con trỏ, là trỏ tới một phần tử nào đó

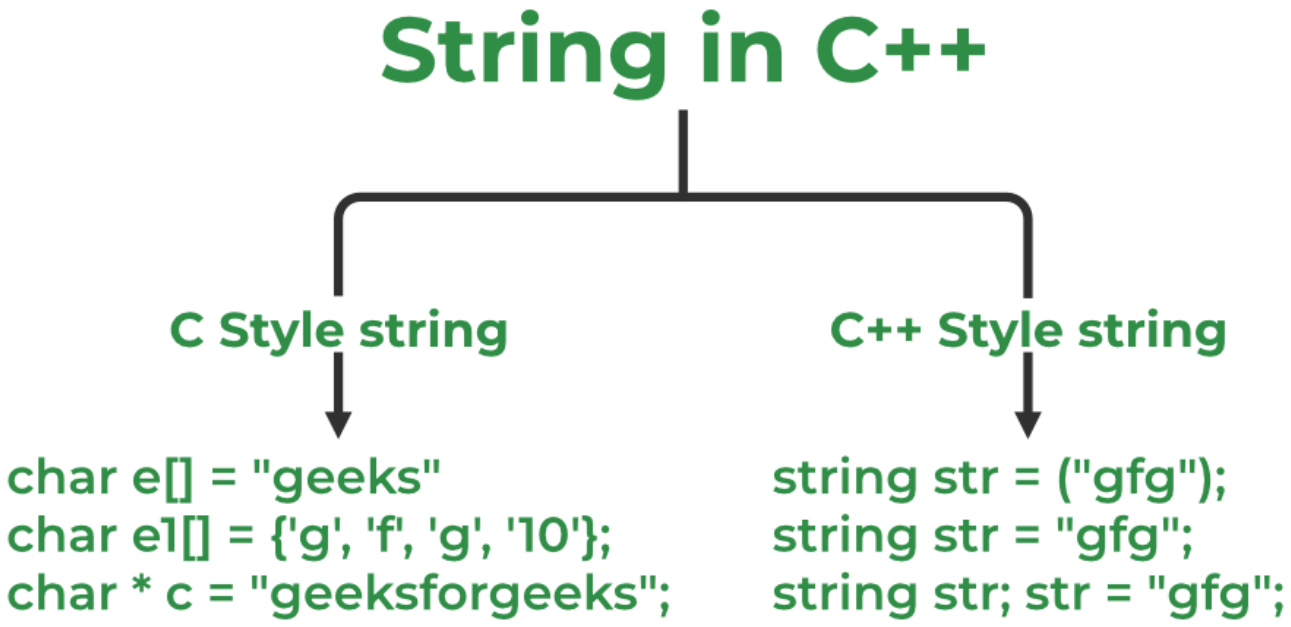
```
int count_x(char* p, char x)
        // count the number of occurrences of x in p[]
        // p is assumed to point to a zero-terminated array of char (or to nothing)
{
        if (p==nullptr) return 0;
        int count = 0;
        for (; *p!=0; ++p)
                if (*p==x)
                        ++count;
        return count;
}
```

chuyển con trỏ trỏ tới phần tử mảng kế tiếp

C++ Strings - Chuỗi

- Là mảng các ký tự **char**, kết thúc bằng **\0** (kiểu chuỗi được định nghĩa trong C)

```
char s[] = {'g', 'f', 'g', '\0'};
char s[4] = {'g', 'f', 'g', '\0'};
char s[4] = "gfg";
char s[] = "gfg";
```



C++ - Kiểu Dữ liệu do Người dùng Xác định

- Kiểu dữ liệu tích hợp sẵn (*built-in types*) được tạo ra từ các kiểu dữ liệu cơ bản, qua khai báo const, hoặc dùng toán tử khai báo.
  - Phong phú nhưng ở cấp thấp, không thuận tiện khi phát triển các ứng dụng thực tế
- C++ dùng khái niệm "trừu tượng" – abstraction – hỗ trợ lập trình viên tạo ra các cấu trúc dữ liệu cấp cao, phức tạp hơn.
  - Thiết kế và triển khai kiểu riêng phù hợp yêu cầu xây dựng ứng dụng
  - Biểu diễn kiểu, mô tả hoạt động lên kiểu được định nghĩa
- Kiểu lớp – *class*
- Kiểu liệt kê - *enumeration*

C++ Structure – Kiểu Cấu trúc

- Tổ chức các thành phần vào cấu trúc dữ liệu với **struct**

```
struct Vector {
        int sz;             // number of elements
        double* elem;       // pointer to elements
};
```

- Khai báo biến của dữ liệu cấu trúc

```
Vector v;
```

C++ Structure – Kiểu Cấu trúc

```
struct Vector {
    int sz;          // number of elements
    double* elem;    // pointer to elements
};
```

- Khởi tạo cấu trúc

```
void vector_init(Vector& v, int s)
{
    v.elem = new double[s];  // allocate an array of s doubles
    v.sz = s;
}
```

- **Vector& v** : truyền **v** bằng tham chiếu, cho phép hàm **vector_init()** sửa đổi nội dung mà **v** trỏ tới
- **new** : toán tử cấp phát vùng nhớ từ bộ nhớ động (*dynamic memory*) và vùng *heap*

C++ Structure – Kiểu Cấu trúc – Cách Sử dụng

- Ví dụ

```
struct Vector {
        int sz;          // number of elements
        double* elem;    // pointer to elements
};
```

```
double read_and_sum(int s)
        // read s integers from cin and return their sum; s is assumed to be positive
{
        Vector v;
        vector_init(v,s);                // allocate s elements for v
        for (int i=0; i!=s; ++i)
                cin>>v.elem[i];          // read into elements

        double sum = 0;
        for (int i=0; i!=s; ++i)
                sum+=v.elem[i];          // take the sum of the elements
        return sum;
}
```

C++ Structure – Kiểu Cấu trúc – Cách Sử dụng

- **.** (dot) : toán tử truy xuất thành phần (members) của cấu trúc qua tên thành phần
- **->** : toán tử truy xuất thành phần cấu trúc bằng con trỏ
- Ví dụ:

```
struct Vector {
    int sz;          // number of elements
    double* elem;    // pointer to elements
};
```

```
void f(Vector v, Vector& rv, Vector* pv)
{
    int i1 = v.sz;        // access through name
    int i2 = rv.sz;       // access through reference
    int i4 = pv->sz;      // access through pointer
}
```

C++ Class – Kiểu lớp

- Tách biệt phần ”giao tiếp” - giao diện – khỏi phần dữ liệu nội bộ của đối tượng, nhằm giữ tính toàn vẹn của dữ liệu trong quá trình sử dụng.
- Kiểu Lớp – *class* : là tập các thành viên (*members*)
- Thành viên: dữ liệu, hàm xử lý, các thành phần kiểu dữ liệu con
- Hai nhóm:
  - **public**: gồm các thành viên xác lập giao diện của lớp
  - **private**: gồm các thành viên chỉ được truy xuất thông qua giao diện

C++ Class – Kiểu lớp – Khai báo

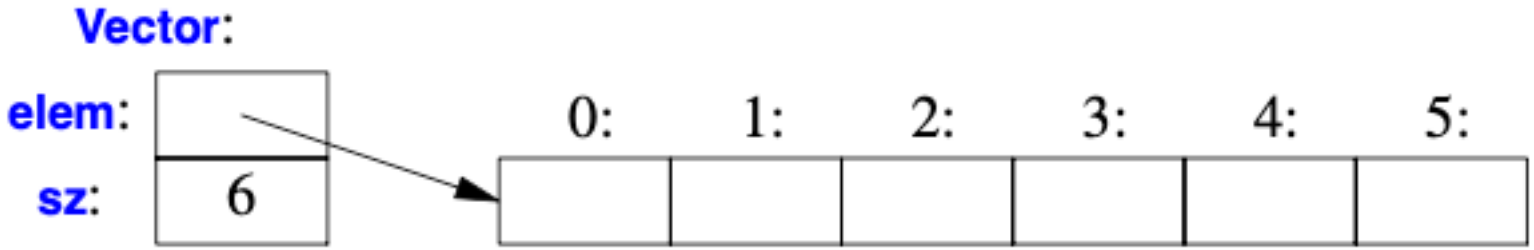
- Khai báo kiểu Lớp

```
class Vector {
public:
        Vector(int s) :elem{new double[s]}, sz{s} { }    // construct a Vector
        double& operator[](int i) { return elem[i]; }    // element access: subscripting
        int size() { return sz; }
private:
        double* elem;  // pointer to the elements
        int sz;        // the number of elements
};
```

C++ Class – Kiểu lớp – Khai báo

- Khai báo biến Lớp

```
Vector v(6);    // a Vector with 6 elements
```



C++ Class – Kiểu lớp – Sử dụng

- Nhập giá trị vào biến Lớp

```
double read_and_sum(int s)
{
        Vector v(s);                        // make a vector of s elements
        for (int i=0; i!=v.size(); ++i)
                cin>>v[i];                  // read into elements

        double sum = 0;
        for (int i=0; i!=v.size(); ++i)
                sum+=v[i];                  // take the sum of the elements
        return sum;
}
```

C++ Class – Kiểu lớp – Constructor

- *Constructor*: “Hàm” được khai báo trùng tên với kiểu lớp
- Để tạo các đối tượng cụ thể của lớp
- Ví dụ: constructor Vector() làm nhiệm vụ tương đương vector_init()
- C++ đảm bảo luôn gọi constructor để tạo đối tượng
  - Tránh việc quên khởi tạo biến đối tượng của lớp

```
public:
    Vector(int s) :elem{new double[s]}, sz{s} { }    // construct a Vector
```

C++ Enumerations – Kiểu Liệt kê

- Cấu trúc liệt kê các giá trị dữ liệu

```
enum class Color { red, blue, green };
enum class Traffic_light { green, yellow, red };

Color col = Color::red;
Traffic_light light = Traffic_light::red;
```

- Được dùng để trình bày một tập con các giá trị số nguyên.
- Viết mã nguồn C++ dễ đọc và giảm lỗi hơn

C++ Enumerations – Kiểu Liệt kê – Phạm vi

- Thành phần liệt kê chỉ thuộc phạm vi của enum class mà nó được khai báo.
  - **Color::red** và **Traffic_light::red** là hai giá trị khác nhau
- Những cách dùng sau sẽ gây lỗi

```
Color x = red;                  // error: which red?
Color y = Traffic_light::red;   // error: that red is not a Color
Color z = Color::red;           // OK

int i = Color::red;             // error: Color::red is not an int
Color c = 2;                    // error: 2 is not a Color
```

C++ Enumerations – Kiểu Liệt kê – Toán tử

- Mặc định: phép gán, phép khởi tạo, phép so sánh
- Định nghĩa các toán tử riêng

```
Traffic_light& operator++(Traffic_light& t)
        // prefix increment: ++
{
        switch (t) {
        case Traffic_light::green:      return t=Traffic_light::yellow;
        case Traffic_light::yellow:     return t=Traffic_light::red;
        case Traffic_light::red:        return t=Traffic_light::green;
        }
}

Traffic_light next = ++light;           // next becomes Traffic_light::green
```

C++ Function – Hàm

C++ - Tính Mô-đun

- Chương trình C++ gồm nhiều phần được phát triển riêng biệt: hàm, kiểu dữ liệu do người dùng định nghĩa, các cấu trúc phân cấp lớp, các mẫu (templates)
- Cần định nghĩa các tương tác kết nối các thành phần
- Khai báo giao diện: xác lập các phần cần có (điều kiện cần và đủ) để sử dụng hàm hoặc kiểu dữ liệu được khai báo

C++ - Tính Mô-đun – Khai báo giao diện hàm

```
double sqrt(double);        // the square root function takes a double and returns a double

class Vector {
public:
        Vector(int s);
        double& operator[](int i);
        int size();
private:
        double* elem;  // elem points to an array of sz doubles
        int sz;
};
```

- Thân hàm hàm sqrt(double) được khai báo ở “đâu đó”, ở đây là trong thư viện toán chuẩn của C++

```
double sqrt(double d)            // definition of sqrt()
{
        // ... algorithm as found in math textbook ...
}
```

C++ - Tính Mô-đun – Khai báo thành viên Lớp

```
Vector::Vector(int s)                    // definition of the constructor
        :elem{new double[s]}, sz{s}      // initialize members
{
}


double& Vector::operator[](int i)        // definition of subscripting
{
        return elem[i];
}

int Vector::size()                       // definition of size()
{
        return sz;
}
```

C++ Biên dịch riêng biệt

- Mã người dùng chỉ thấy các khai báo kiểu và hàm mà được sử dụng.
- Các định nghĩa về các loại và hàm đó nằm trong các tệp nguồn riêng biệt và được biên dịch riêng.
- Tổ chức một chương trình thành tập hợp các đoạn mã bán độc lập.
  - Giảm thời gian biên dịch
  - Thực hiện phân tách có logic chương trình thành các thành phần → giảm lỗi
- Thư viện thường được biên dịch riêng biệt

C++ Biên dịch riêng biệt – Chia tách tập tin giao diện

- Gom nhóm các khai báo xác lập giao diện của một mô-đun vào cùng một tập tin (***header*** file) có tên gợi nhớ tới mục đích sử dụng của nhóm này.
- Ví dụ: file Vector.h

```
// Vector.h:

class Vector {
public:
        Vector(int s);
        double& operator[](int i);
        int size();
private:
        double* elem;        // elem points to an array of sz doubles
        int sz;
};
```

C++ Biên dịch riêng biệt – Sử dụng tập tin giao diện

- Sử dụng: *include* header file để truy xuất vào interface.

```
// user.cpp:

#include "Vector.h"        // get Vector's interface
#include <cmath>           // get the the standard-library math function interface including sqrt()
using namespace std;       // make std members visible

double sqrt_sum(Vector& v)
{
        double sum = 0;
        for (int i=0; i!=v.size(); ++i)
                sum+=sqrt(v[i]);                    // sum of square roots
        return sum;
}
```

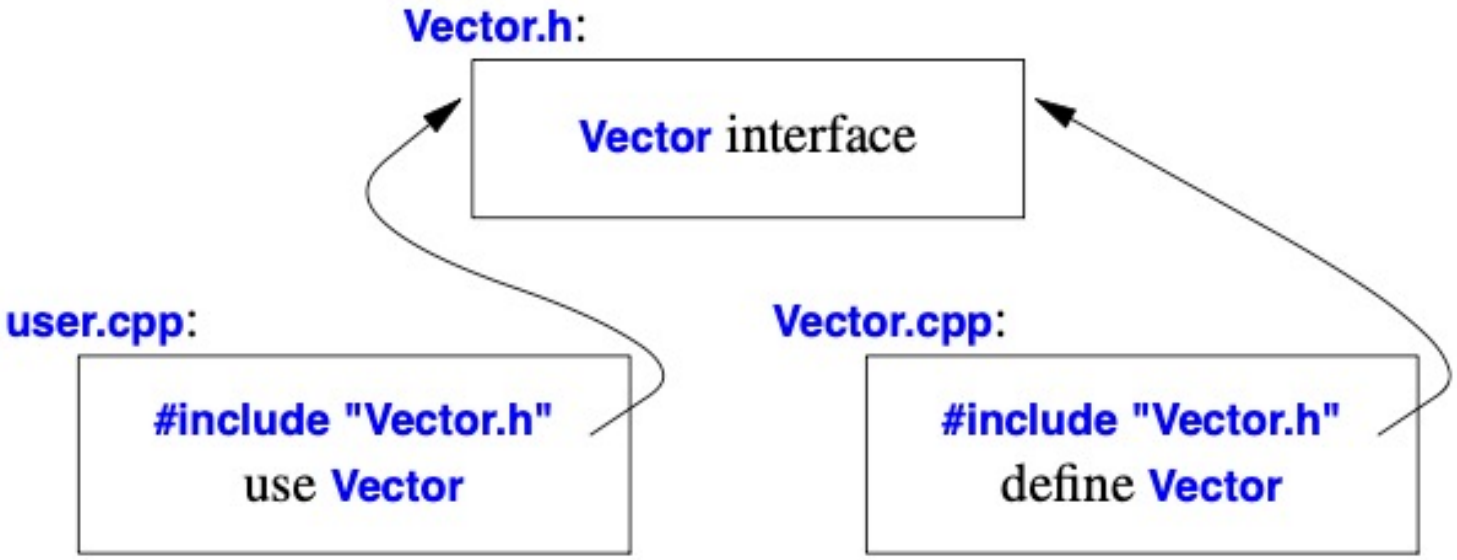
C++ Biên dịch riêng biệt – Sử dụng tập tin giao diện

- Sử dụng: *include* header file để truy xuất vào interface.

```
// Vector.cpp:

#include "Vector.h" // get the interface

Vector::Vector(int s)
        :elem{new double[s]}, sz{s}
{
}

double& Vector::operator[](int i)
{
        return elem[i];
}

int Vector::size()
{
        return sz;
}
```

C++ Biên dịch riêng biệt – Sử dụng tập tin giao diện

- Sử dụng: *include* header file để truy xuất vào interface.



C++ - Namespace

- Khai báo trong namespace các tên để tránh bị xung đột với các tên khác

```
namespace My_code {
        class complex { /* ... */ };
        complex sqrt(complex);
        // ...
        int main();
}

int My_code::main()
{
        complex z {1,2};
        auto z2 = sqrt(z);
        std::cout << '{' << z2.real() << ',' << z2.imag() << "}\n";
        // ...
};

int main()
{
        return My_code::main();
}
```

- Sử dụng các tên đã khai báo trong namespace của thư viên chuẩn

```
using namespace std;
```

Xử lý Lỗi trong C++ - Xử lý Ngoại lệ (Exceptions)

- Ví dụ: sử dụng lớp Vector

- Thử truy xuất phần tử nằm ngoài phạm vi của một vector?
  - Do không lường trước chương trình sẽ sử dụng Vector và sử dụng như thế nào
  - Người dùng không chuẩn bị để phát hiện sự cố
- Giải pháp: Vector phải được viết để có thể phát hiện các cố gắng truy xuất phần tử ngoài phạm vi và phát cảnh báo

Xử lý Lỗi trong C++ - Xử lý Ngoại lệ (Exceptions)

- "Bắt" ngoại lệ

```
double& Vector::operator[](int i)
{
        if (i<0 || size()<=i) throw out_of_range{"Vector::operator[]"};
        return elem[i];
}
```

Xử lý Lỗi trong C++ - Xử lý Ngoại lệ (Exceptions)

- Trả cảnh báo ngoại lệ cho hàm gọi thực thi Vector để xử lý ngoại lệ

```
void f(Vector& v)
{
        // ...
        try { // exceptions here are handled by the handler defined below

                v[v.size()] = 7;  // try to access beyond the end of v
        }
        catch (out_of_range) {     // oops: out_of_range error
                // ... handle range error ...
        }
        // ...
}
```

Xử lý Lỗi trong C++ - Tính Bất biến (Invariants)

- Các mô tả, và theo đó là cài đặt đã không ràng buộc hết các trường hợp gây ra lỗi.
- Ví dụ:

```
Vector v(–27);
```

- Giải pháp: kiểm tra giá trị đầu vào thỏa mãn các điều kiện ràng buộc mới cho phép tiếp tục xử lý sau đó

```
Vector::Vector(int s)
{
        if (s<0) throw length_error{};
        elem = new double[s];
        sz = s;
}
```

Xử lý Lỗi trong C++ - Tính Bất biến (Invariants)

- Kiểm tra cả khả năng xảy ra lỗi trong việc cấp phát bộ nhớ.

```
void test()
{
        try {
                Vector v(–27);
        }
        catch (std::length_error) {
                // handle negative size
        }
        catch (std::bad_alloc) {
                // handle memory exhaustion
        }
}
```

Xử lý Lỗi trong C++ - Đánh giá tĩnh (Static Assertions)

- Ngoại lệ chỉ báo các lỗi xảy ra trong quá trình thực thi chương trình (run-time errors)
- Cần kiểm tra, phát hiện và xử lý cả lỗi xảy ra trong quá trình biên dịch.
- Viết các đoạn lệnh dùng **static_assert** kiểm tra, đánh giá.

- **static_assert(A, S)** in ra thông báo **S** báo lỗi biên dịch nếu điều kiện **A** không đúng.

- Ví dụ:

```
static_assert(4<=sizeof(int), "integers are too small");   // check integer size
```

Xử lý Lỗi trong C++ - Đánh giá tĩnh (Static Assertions)

- Ví dụ khác:

```
constexpr double C = 299792.458;                    // km/s

void f(double speed)
{
        const double local_max = 160.0/(60*60);     // 160 km/h == 160.0/(60*60) km/s

        static_assert(speed<C,"can't go that fast");    // error: speed must be a constant
        static_assert(local_max<C,"can't go that fast");  // OK

        // ...
}
```

Hết phần này